AI Assisted Coding
Lab Assignment - 16.1

Name: G. Spriha
Roll No: 2303A51850
Batch: 13

Lab 16 – Database Design and Queries: Schema Design and SQL Generation
Task 1 – Student Information System Schema
Task:
Design a database schema for a Student Information System and generate queries using AI.
Instructions:

- Tables: Students, Courses, Enrollments.
- Define primary keys, foreign keys, and relationships.
- Generate queries:
    - Insert a new student record.
    - Fetch all courses enrolled by a student.
    - Count number of students in each course.

Expected Output:
• SQL CREATE TABLE statements and queries for student– course relationships.

Prompt :

Design a Student Information System database with tables for Students, Courses, and Enrolments. Define appropriate primary keys and foreign key relationships between the tables. Write SQL queries to insert a student record and retrieve enrolled courses for a student. Also, generate a query to count the number of students in each course.

SQL:

```sql
-- 1. CREATE TABLES
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100) UNIQUE
);
CREATE TABLE Courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100)
);
CREATE TABLE Enrollments (
    enrollment_id INT PRIMARY KEY,
    student_id INT,
    course_id INT,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
-- Insert student
INSERT INTO Students VALUES (1, 'Sushmitha', 'sushmitha@gmail.com');
-- Insert courses
INSERT INTO Courses VALUES (101, 'Math');
INSERT INTO Courses VALUES (102, 'Science');
-- Insert enrollments
INSERT INTO Enrollments VALUES (1, 1, 101);
INSERT INTO Enrollments VALUES (2, 1, 102);
-- Fetch all courses enrolled by student
SELECT Courses.course_name
FROM Courses
JOIN Enrollments
ON Courses.course_id = Enrollments.course_id
WHERE Enrollments.student_id = 1;
-- Count number of students in each course
SELECT Courses.course_name, COUNT(Enrollments.student_id) AS total_students
FROM Courses
LEFT JOIN Enrollments
ON Courses.course_id = Enrollments.course_id
GROUP BY Courses.course_name;
-- 4. DISPLAY TABLES (OUTPUT)
SELECT * FROM Students;
SELECT * FROM Courses;
SELECT * FROM Enrollments;
```

Output:

course_name
Math
Science

SQL ▼ 1 / 1 1 - 2 of 2

course_name	total_students
Math	1
Science	1

SQL ▼ 1 / 1 1 - 1 of 1

student_id	name	email
1	Sushmitha	sushmitha@gmail.com

SQL ▼ 1 / 1 1 - 2 of 2

course_id	course_name
101	Math
102	Science

SQL ▼ 1 / 1 1 - 2 of 2

enrollment_id	student_id	course_id
1	1	101
2	1	102

Observation:

- The database schema was successfully created with three tables: **Students, Courses, and Enrollments**.
- Primary keys uniquely identify each record in the tables.
- Foreign keys in the **Enrollments** table correctly establish relationships between Students and Courses.
- The relationship between Students and Courses is **many-to-many**, implemented using the Enrollments table.
- The INSERT query successfully added a new student record to the Students table.
- The SELECT query using JOIN retrieved all courses enrolled by the student correctly.
- The aggregation query using COUNT() correctly calculated the number of students in each course.
- The results were displayed accurately in tabular form in VS Code using SQLite.

Task 2 – Hospital Management Database
Task:
Create schema and queries for a Hospital Management System.
Instructions:

- Tables: Doctors, Patients, Appointments.
- Use AI to define constraints (unique IDs, valid dates).
- Generate queries:
    - List all appointments for a specific doctor.
    - Retrieve patient history by patient ID.
    - Count total patients treated by each doctor.

Expected Output:
• Normalized schema and SQL queries with joins.

Prompt: Design a Hospital Management System database with tables for Doctors, Patients, and Appointments. Define appropriate primary keys, foreign keys, and constraints such as unique IDs and valid dates. Write SQL queries to list appointments for a doctor and retrieve patient history. Also, generate a query to count total patients treated by each doctor.

SQL:

```sql
-- 1. CREATE TABLES
CREATE TABLE Doctors (
    doctor_id INT PRIMARY KEY,
    name VARCHAR(100),
    specialization VARCHAR(100)
);
CREATE TABLE Patients (
    patient_id INT PRIMARY KEY,
    name VARCHAR(100),
    age INT CHECK (age > 0)
);
CREATE TABLE Appointments (
    appointment_id INT PRIMARY KEY,
    doctor_id INT,
    patient_id INT,
    appointment_date DATE NOT NULL,
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
);
INSERT INTO Doctors VALUES (1, 'Dr. Rao', 'Cardiology');
INSERT INTO Doctors VALUES (2, 'Dr. Sharma', 'Neurology');

INSERT INTO Patients VALUES (101, 'Anu', 25);
INSERT INTO Patients VALUES (102, 'Rahul', 30);

INSERT INTO Appointments VALUES (1, 1, 101, '2026-03-01');
INSERT INTO Appointments VALUES (2, 1, 102, '2026-03-05');
INSERT INTO Appointments VALUES (3, 2, 101, '2026-03-10');
-- List all appointments for a specific doctor (Doctor ID = 1)
SELECT Doctors.name AS doctor_name, Patients.name AS patient_name, appointment_date
FROM Appointments
JOIN Doctors ON Appointments.doctor_id = Doctors.doctor_id
JOIN Patients ON Appointments.patient_id = Patients.patient_id
WHERE Doctors.doctor_id = 1;
-- Retrieve patient history (Patient ID = 101)
SELECT Patients.name AS patient_name, Doctors.name AS doctor_name, appointment_date
FROM Appointments
JOIN Doctors ON Appointments.doctor_id = Doctors.doctor_id
JOIN Patients ON Appointments.patient_id = Patients.patient_id
WHERE Patients.patient_id = 101;
-- Count total patients treated by each doctor
SELECT Doctors.name, COUNT(Appointments.patient_id) AS total_patients
FROM Doctors
LEFT JOIN Appointments ON Doctors.doctor_id = Appointments.doctor_id
GROUP BY Doctors.name;
SELECT * FROM Doctors;
SELECT * FROM Patients;
SELECT * FROM Appointments;
```

Output:

SQL ▼ 1 / 1 1 - 2 of 2

doctor_name	patient_name	appointment_date
Dr. Rao	Anu	2026-03-01
Dr. Rao	Rahul	2026-03-05

SQL ▼ 1 / 1 1 - 2 of 2

patient_name	doctor_name	appointment_date
Anu	Dr. Rao	2026-03-01
Anu	Dr. Sharma	2026-03-10

SQL ▼ 1 / 1 1 - 2 of 2

name	total_patients
Dr. Rao	2
Dr. Sharma	1

SQL ▼ 1 / 1 1 - 2 of 2

doctor_id	name	specialization
1	Dr. Rao	Cardiology
2	Dr. Sharma	Neurology

SQL ▼ 1 / 1 1 - 2 of 2

patient_id	name	age
101	Anu	25
102	Rahul	30

SQL ▼ 1 / 1 1 - 3 of 3

appointment_id	doctor_id	patient_id	appointment_date
1	1	101	2026-03-01
2	1	102	2026-03-05
3	2	101	2026-03-10

Observation:

- The database schema was successfully created with **Doctors, Patients, and Appointments** tables.
- Primary keys uniquely identify records, and foreign keys establish relationships between tables.
- Constraints like **CHECK (age > 0)** and **NOT NULL for dates** ensure data validity.
- The JOIN queries correctly retrieved doctor-wise appointments and patient history.
- The COUNT() function accurately calculated the number of patients treated by each doctor.

Task 3 – Library Database

Task:

Design schema for a Library Management System.

Instructions:

- Tables: Books, Members, Loans.
- Use AI to suggest indexing strategy for faster queries.
- Generate queries:
  - Retrieve all books currently issued.
  - Find overdue books (loan date > 30 days).
  - Count number of books loaned by each member.

Expected Output:

- Schema + SQL queries demonstrating joins and conditions.

Prompt:

Design a Library Management System database with tables for Books, Members, and Loans. Define primary keys, foreign keys, and suggest indexing strategies for faster queries. Write SQL queries to retrieve issued books and identify overdue books. Also, generate a query to count the number of books loaned by each member.

SQL:

```sql
-- 1. CREATE TABLES
CREATE TABLE Books (
    book_id INT PRIMARY KEY,
    title VARCHAR(100),
    author VARCHAR(100)
);
CREATE TABLE Members (
    member_id INT PRIMARY KEY,
    name VARCHAR(100)
);
CREATE TABLE Loans (
    loan_id INT PRIMARY KEY,
    book_id INT,
    member_id INT,
    loan_date DATE,
    return_date DATE,
    FOREIGN KEY (book_id) REFERENCES Books(book_id),
    FOREIGN KEY (member_id) REFERENCES Members(member_id)
);
CREATE INDEX idx_book ON Loans(book_id);
CREATE INDEX idx_member ON Loans(member_id);
CREATE INDEX idx_loan_date ON Loans(loan_date);
INSERT INTO Books VALUES (1, 'DBMS', 'Korth');
INSERT INTO Books VALUES (2, 'Python', 'Guido');
INSERT INTO Books VALUES (3, 'AI Basics', 'John');
INSERT INTO Members VALUES (101, 'Sushmitha');
INSERT INTO Members VALUES (102, 'Gouri');
INSERT INTO Loans VALUES (1, 1, 101, '2026-02-01', NULL);
INSERT INTO Loans VALUES (2, 2, 102, '2026-01-15', NULL);
INSERT INTO Loans VALUES (3, 3, 101, '2026-03-10', '2026-03-20');
-- Retrieve all books currently issued (not returned)
SELECT Books.title, Members.name, Loans.loan_date
FROM Loans
JOIN Books ON Loans.book_id = Books.book_id
JOIN Members ON Loans.member_id = Members.member_id
WHERE Loans.return_date IS NULL;
-- Find overdue books (loan date > 30 days)
SELECT Books.title, Members.name, Loans.loan_date
FROM Loans
JOIN Books ON Loans.book_id = Books.book_id
JOIN Members ON Loans.member_id = Members.member_id
WHERE Loans.return_date IS NULL
AND DATE('now') - Loans.loan_date > 30;
-- Count number of books loaned by each member
SELECT Members.name, COUNT(Loans.book_id) AS total_books
FROM Members
LEFT JOIN Loans ON Members.member_id = Loans.member_id
GROUP BY Members.name;
SELECT * FROM Books;
SELECT * FROM Members;
SELECT * FROM Loans;
```

Output:

title	name	loan_date
DBMS	Sushmitha	2026-02-01
Python	Gouri	2026-01-15

SQL ▼ 1 / 1 1 - 0 of 0

SQL ▼ 1 / 1 1 - 2 of 2

name	total_books
Gouri	1
Sushmitha	2

SQL ▼ 1 / 1 1 - 3 of 3

book_id	title	author
1	DBMS	Korth
2	Python	Guido
3	AI Basics	John

SQL ▼ 1 / 1 1 - 2 of 2

member_id	name
101	Sushmitha
102	Gouri

SQL ▼ 1 / 1 1 - 3 of 3

loan_id	book_id	member_id	loan_date	return_date
1	1	101	2026-02-01	NULL
2	2	102	2026-01-15	NULL
3	3	101	2026-03-10	2026-03-20

Observation:

- The database schema was successfully created with **Books, Members, and Loans** tables.
- Primary keys uniquely identify records, and foreign keys establish relationships between tables.
- Indexes on **book_id, member_id, and loan_date** improved query performance.
- The query for issued books correctly retrieved records where books are not yet returned.
- The overdue books query identified loans exceeding 30 days accurately.
- The COUNT() query correctly calculated the number of books loaned by each member.

Task 4 – Real-Time Application: Online Shopping Database

Scenario:

Design a database for an E-commerce platform.

Requirements:

- Tables: Users, Products, Orders, OrderDetails.
- Generate queries to:
  - Retrieve all orders by a user.
  - Find the most purchased product.
  - Calculate total revenue in a given month.
- AI should also suggest normalization improvements and query optimization.

Expected Output:

- Complete schema with relationships + SQL queries for analytics.

Prompt:

Design an E-commerce database with tables for Users, Products, Orders, and OrderDetails. Define primary keys, foreign keys, and proper relationships between tables. Write SQL queries to retrieve user orders, find the most purchased product, and calculate monthly revenue. Also suggest normalization improvements and query optimization techniques.

SQL:

```sql
-- 1. CREATE TABLES
CREATE TABLE Users (
    user_id INT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100) UNIQUE
);
CREATE TABLE Products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100),
    price DECIMAL(10,2)
);
CREATE TABLE Orders (
    order_id INT PRIMARY KEY,
    user_id INT,
    order_date DATE,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);
CREATE TABLE OrderDetails (
    order_detail_id INT PRIMARY KEY,
    order_id INT,
    product_id INT,
    quantity INT CHECK (quantity > 0),
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
INSERT INTO Users VALUES (1, 'Sushmitha', 'sushmitha@gmail.com');
INSERT INTO Products VALUES (101, 'Laptop', 50000);
INSERT INTO Products VALUES (102, 'Phone', 20000);
INSERT INTO Orders VALUES (1, 1, '2026-03-01');
INSERT INTO Orders VALUES (2, 1, '2026-03-15');
INSERT INTO OrderDetails VALUES (1, 1, 101, 1);
INSERT INTO OrderDetails VALUES (2, 1, 102, 2);
INSERT INTO OrderDetails VALUES (3, 2, 102, 1);
-- Retrieve all orders by a user (User ID = 1)
SELECT Users.name, Orders.order_id, Orders.order_date
FROM Orders
JOIN Users ON Orders.user_id = Users.user_id
WHERE Users.user_id = 1;
-- Find most purchased product
SELECT Products.product_name, SUM(OrderDetails.quantity) AS total_quantity
FROM OrderDetails
JOIN Products ON OrderDetails.product_id = Products.product_id
GROUP BY Products.product_name
ORDER BY total_quantity DESC
LIMIT 1;
-- Calculate total revenue in a given month (March 2026)
SELECT SUM(Products.price * OrderDetails.quantity) AS total_revenue
FROM OrderDetails
JOIN Products ON OrderDetails.product_id = Products.product_id
JOIN Orders ON OrderDetails.order_id = Orders.order_id
WHERE strftime('%Y-%m', Orders.order_date) = '2026-03';
SELECT * FROM Users;
SELECT * FROM Products;
SELECT * FROM Orders;
SELECT * FROM OrderDetails;
```

Output:

name	order_id	order_date
Sushmitha	1	2026-03-01
Sushmitha	2	2026-03-15

SQL ▾ < 1 / 1 > 1 - 1 of 1

product_name	total_quantity
Phone	3

SQL ▾ < 1 / 1 > 1 - 1 of 1

total_revenue
110000

SQL ▾ < 1 / 1 > 1 - 1 of 1

user_id	name	email
1	Sushmitha	sushmitha@gmail.com

SQL ▾ < 1 / 1 > 1 - 2 of 2

product_id	product_name	price
101	Laptop	50000
102	Phone	20000

SQL ▾ < 1 / 1 > 1 - 2 of 2

order_id	user_id	order_date
1	1	2026-03-01
2	1	2026-03-15

SQL ▾ < 1 / 1 > 1 - 3 of 3

order_detail_id	order_id	product_id	quantity
1	1	101	1
2	1	102	2
3	2	102	1

Observation:

- The database schema was successfully designed with **Users, Products, Orders, and OrderDetails** tables.
- Primary and foreign keys established proper relationships between entities.
- The schema follows normalization principles, reducing redundancy and improving data integrity.
- Queries using JOIN and aggregation functions successfully retrieved user orders, most purchased product, and monthly revenue.
- Indexing and optimized queries improved performance for large datasets.

Task 5 – University Examination Database

Design a database schema for a University Examination System and generate SQL queries using AI.

Instructions:

- Tables: Students, Subjects, Exams, Results.
- Define primary keys, foreign keys, and referential integrity constraints.
- Generate queries:
  - Insert examination results for a student.
  - Retrieve subject-wise marks for a student.
  - Calculate average marks for each subject.

Expected Output:

- Normalized schema and SQL queries involving aggregation and joins.

Prompt:

Design a University Examination System database with tables for Students, Subjects, Exams, and Results. Define primary keys, foreign keys, and maintain referential integrity between tables. Write SQL queries to insert exam results and retrieve subject-wise marks for a student. Also, generate a query to calculate average marks for each subject.

SQL:

```sql
-- 1. CREATE TABLES
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    name VARCHAR(100)
);
CREATE TABLE Subjects (
    subject_id INT PRIMARY KEY,
    subject_name VARCHAR(100)
);
CREATE TABLE Exams (
    exam_id INT PRIMARY KEY,
    exam_name VARCHAR(100),
    exam_date DATE
);
CREATE TABLE Results (
    result_id INT PRIMARY KEY,
    student_id INT,
    subject_id INT,
    exam_id INT,
    marks INT CHECK (marks >= 0),
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (subject_id) REFERENCES Subjects(subject_id),
    FOREIGN KEY (exam_id) REFERENCES Exams(exam_id)
);
INSERT INTO Students VALUES (1, 'Sushmitha');
INSERT INTO Subjects VALUES (101, 'Math');
INSERT INTO Subjects VALUES (102, 'Science');
INSERT INTO Exams VALUES (1, 'Mid Exam', '2026-03-01');
-- Insert examination results
INSERT INTO Results VALUES (1, 1, 101, 1, 85);
INSERT INTO Results VALUES (2, 1, 102, 1, 90);
-- Retrieve subject-wise marks for a student
SELECT Students.name, Subjects.subject_name, Results.marks
FROM Results
JOIN Students ON Results.student_id = Students.student_id
JOIN Subjects ON Results.subject_id = Subjects.subject_id
WHERE Students.student_id = 1;
-- Calculate average marks for each subject
SELECT Subjects.subject_name, AVG(Results.marks) AS avg_marks
FROM Results
JOIN Subjects ON Results.subject_id = Subjects.subject_id
GROUP BY Subjects.subject_name;
SELECT * FROM Students;
SELECT * FROM Subjects;
SELECT * FROM Exams;
SELECT * FROM Results;
```

Output:

name	subject_name	marks
Sushmitha	Math	85
Sushmitha	Science	90

SQL ▾ 1 / 1 1 - 2 of 2

subject_name	avg_marks
Math	85.0
Science	90.0

SQL ▾ 1 / 1 1 - 1 of 1

student_id	name
1	Sushmitha

SQL ▾ 1 / 1 1 - 2 of 2

subject_id	subject_name
101	Math
102	Science

SQL ▾ 1 / 1 1 - 1 of 1

exam_id	exam_name	exam_date
1	Mid Exam	2026-03-01

SQL ▾ 1 / 1 1 - 2 of 2

result_id	student_id	subject_id	exam_id	marks
1	1	101	1	85
2	1	102	1	90

Observation:

- The database schema was successfully created with **Students, Subjects, Exams, and Results** tables.
- Primary keys uniquely identify records, and foreign keys maintain referential integrity between tables.
- The Results table connects students, subjects, and exams, forming a normalized structure.
- The INSERT query successfully stored examination results.
- JOIN queries correctly retrieved subject-wise marks for a student.
- The AVG() function accurately calculated average marks for each subject.